Task D: Validation and Algorithm Analysis

1. Algorithm Design

Task A: Closest Delivery Locations

Task A uses a divide-and-conquer algorithm to find the closest pair of delivery locations.

First, the points are sorted by their x-coordinate. The sorted points are divided into a left half and a right half. The algorithm recursively finds the closest pair in each half. It then keeps the smaller distance found between the two halves.

The algorithm also checks a vertical strip around the dividing line. This strip is necessary because the closest pair may contain one point from the left half and one point from the right half. The points in the strip are already sorted by their y-coordinate, so each point only needs to be compared with a small constant number of nearby points.

For subproblems containing three or fewer points, the algorithm uses brute force and compares every pair directly.

Task B: Counting Dispatch-Order Inversions

Task B uses merge sort to count inversions efficiently.

First, the actual dispatch order is converted into an array of planned-order ranks. For example, if the planned order is [A, B, C, D] and the actual order is [A, C, D, B], the rank array is [1, 3, 4, 2].

The algorithm recursively divides the rank array into left and right halves. It counts inversions in each half and then counts cross-half inversions while merging the two sorted halves.

When an element from the right half is smaller than an element from the left half, it forms an inversion with all remaining elements in the left half.

Task C: Selecting a Delivery-Time Threshold

Task C uses the deterministic median-of-medians selection algorithm to find the kth smallest value without sorting the entire array.

The input is divided into groups of at most five values. Each small group is sorted, and its median is selected. The algorithm recursively finds the median of these group medians and uses it as the pivot.

The values are partitioned into three groups:

- Values less than the pivot
- Values equal to the pivot
- Values greater than the pivot

The algorithm only recurses into the group containing the desired kth position. It discards the other groups because they cannot contain the answer.

2. Time and Space Complexity

Task A: Closest Delivery Locations

The closest-pair algorithm begins by sorting the points by their x-coordinate and y-coordinate. This takes $O(n \log n)$ time.

At each recursive level, the points are divided into two halves, and the strip is checked in linear time. The strip check does not compare every possible pair. Because the strip is sorted by y-coordinate, each point is compared with only a constant number of nearby points.

The recurrence is: T(n) = 2T(n/2) + O(n)

Therefore, the total worst-case running time is: O(n log n)

The auxiliary space is O(n), because the algorithm stores sorted lists, left and right sublists, and the strip during recursion.

Task B: Counting Dispatch-Order Inversions

The inversion-counting algorithm first converts the actual delivery order into planned-order ranks. This takes O(n) time.

The main algorithm uses merge sort. It recursively divides the array into two halves, counts inversions in each half, and counts cross-half inversions while merging.

The recurrence is: T(n) = 2T(n/2) + O(n)

Therefore, the running time is: O(n log n)

The auxiliary space is O(n), because the merge process requires temporary arrays to store the sorted halves.

Task C: Selecting a Delivery-Time Threshold

The selection algorithm uses the deterministic median-of-medians method. The input is divided into groups of at most five elements, and each small group is sorted to find its median.

The median of the group medians is used as the pivot. This produces a sufficiently balanced partition, so a constant fraction of the elements can be discarded during each recursive step.

The recurrence is approximately: T(n) = T(n/5) + T(7n/10) + O(n)

Therefore, the worst-case running time is: O(n)

The auxiliary space is $O(n)$, because the implementation creates lists for the groups, medians, and the three partitions: values less than, equal to, and greater than the pivot.

The deterministic pivot is important because it guarantees linear worst-case performance. A poor pivot could create very unbalanced partitions and lead to slower performance.

3. Correctness Testing

Each implementation was tested using the required test cases from the assignment and additional edge cases.

For Task A, six tests were run. These included the four required cases, a case with multiple tied closest pairs, and an invalid input case containing fewer than two points. The tests verified both the returned squared distance and the IDs of the closest pair.

For Task B, five tests were run. These included the four required cases and the large reversed-order case with 100,000 delivery IDs. The algorithm correctly returned 4,999,950,000 inversions for the reversed input.

For Task C, eight tests were run. These included the five required cases, an already sorted input, a reverse-sorted input, and an input containing many duplicate values.

All 19 automated tests passed successfully.

The simple reference methods used for manual verification were:

- Task A: Comparing every pair of points using brute force
- Task B: Comparing every pair of positions using nested loops
- Task C: Sorting a copy of the input and selecting position k - 1

4. Performance Evidence

I measured the execution time of each algorithm using input sizes of 100, 1000, and 5000 elements. The times are approximate and measured in milliseconds.

Input size (n)	Task A (ms)	Task B (ms)	Task C (ms)
100	0.421	0.130	0.019
1,000	6.366	1.609	0.345
5,000	33.879	9.737	2.490

The observed results match the expected time complexities:

- Task A: Runtime increased from 0.421 ms at n = 100 to 33.879 ms at n = 5,000. This growth is consistent with the expected O(n log n) complexity.
- Task B: Runtime increased from 0.130 ms to 9.737 ms. The increase is also consistent with O(n log n) merge-sort behavior.
- Task C: Runtime increased from 0.019 ms to 2.490 ms. The results show approximately linear growth overall, which supports the expected O(n) worst-case complexity.